

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования «Уральский федеральный университет имени первого Президента
России Б. Н. Ельцина»

Лабораторная работа №1

Выполнила: студент 1 курса магиструры
“Прикладной анализ данных”
Мансурова Алина Рамильевна

Екатеринбург 2025

Цель работы

Освоить базовые принципы работы с платформой контейнеризации Docker: научиться запускать и управлять контейнерами, настраивать постоянное хранение данных с помощью томов, организовывать взаимодействие между сервисами через пользовательские сети, а также автоматизировать развёртывание многокомпонентного приложения с использованием Docker Compose.

Задачи работы

1. Установить и проверить работоспособность Docker с помощью образа hello-world.
2. Изучить команды управления контейнерами: запуск (`docker run`), просмотр (`docker ps`, `docker ps -a`), остановка (`docker stop`) и удаление (`docker rm`).
3. Развернуть веб-сервер Nginx в контейнере и обеспечить доступ к нему через браузер.
4. Запустить контейнер с системой управления базами данных PostgreSQL, настроить его с помощью переменных окружения и подключиться к нему через консольный клиент `psql`.
5. Создать и использовать Docker-том для обеспечения постоянного хранения данных PostgreSQL.
6. Запустить веб-интерфейс pgAdmin в отдельном контейнере и настроить подключение к экземпляру PostgreSQL.
7. Организовать взаимодействие между контейнерами (PostgreSQL и pgAdmin) с помощью пользовательской Docker-сети.
8. Автоматизировать развёртывание всей инфраструктуры (Nginx, PostgreSQL, pgAdmin) путём создания и валидации файла `docker-compose.yml`.

Часть 0: Установка и проверка Docker

Команды:

```
docker --version
docker compose --version
docker run hello-world
```

```
docker -- version
```

```
Windows PowerShell
Рекомендуется установить: Поддержка Windows ans Linux
Вспомогательная информация: powershell.com/ans.
PS C:\Users\Alina> docker --version
Client:
Version:           28.4.0
API version:       1.51
Go version:        go1.24.7
Git commit:        d8b46d5
Built:             Wed Sep 1 20:59:40 2025
OS/Arch:           windows/amd64
Context:           desktop-linux

Server: Docker Desktop 4.46.0 (206640)
Engine:
Version:           28.4.0
API version:       1.51 (minimum version 1.24)
Go version:        go1.24.7
Git commit:        2d66b79
Built:             Wed Sep 1 20:57:37 2025
OS/Arch:           linux/amd64
Experimental:      false
Containers:
Version:           1.7.27
GitCommit:         8f984ec8b9a752232cad45f0827c8b3437aac3f4da
runc:
Version:           1.2.5
GitCommit:         v1.2.5-0-g5f923ef
docker-init:
Version:           0.19.0
GitCommit:         de40a0a
PS C:\Users\Alina>
```

```
docker run hello-world
```

```
Windows PowerShell
PS C:\Users\Alina> docker run --rm --name hello-world --pull=always hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

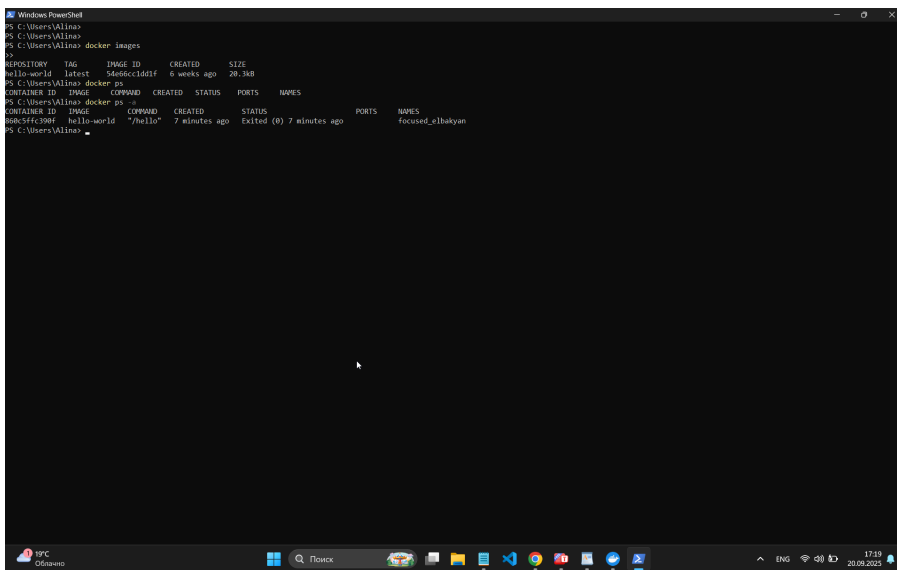
PS C:\Users\Alina>
```

Часть 1: Базовые команды Docker. Работа с образами и контейнерами

Просмотр информации: в

```
docker images
docker ps
docker ps -a
```

Выполнение:



```
PS C:\Users\Allina>
PS C:\Users\Allina>
PS C:\Users\Allina> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
hello-world         latest             560dc1d1f          6 weeks ago        20.3kB
PS C:\Users\Allina> docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
PS C:\Users\Allina> docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
80b5f4c390f        hello-world        "/hello"            7 minutes ago       Exited (0) 7 minutes ago              focused_elbakyan
PS C:\Users\Allina>
```

Запуск простого контейнера (на примере Nginx)

```
docker run -d -p 8080:80 --name my_webtestservak nginx:alpine
```

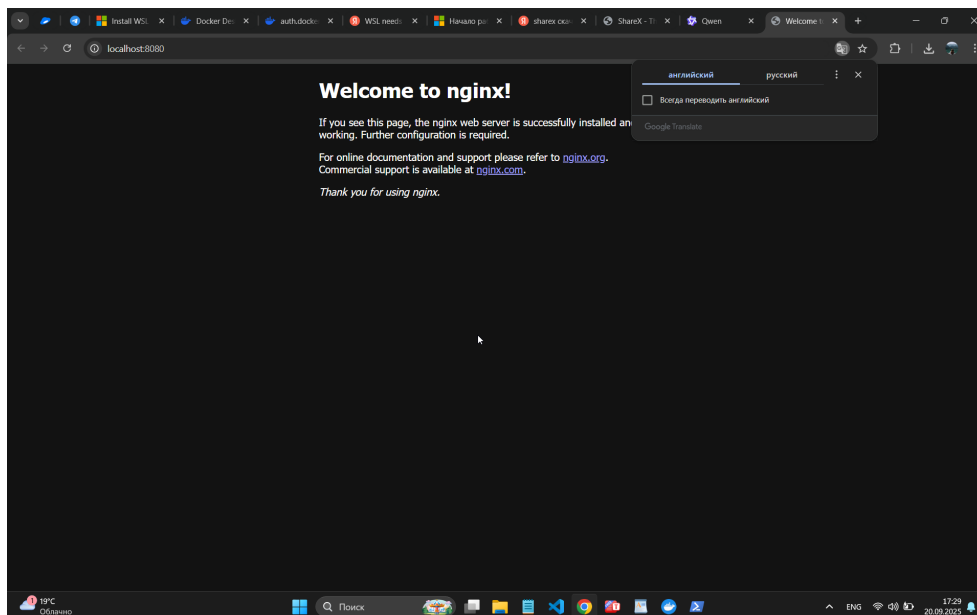
Выполнение

```
Windows PowerShell
docker: invalid reference format

Run 'docker run --help' for more information
PS C:\Users\Alina> docker run -d -p 8080:80 --name my_webtestservak nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
sha1ff4080f82: Pull complete
sha2ef2a2c: Pull complete
a992fbc61acc: Pull complete
aafbbe09837: Pull complete
d6c52a340cc: Pull complete
9824c27679d3: Pull complete
908e1f23537: Pull complete
7ab8d6741e18: Pull complete
Digest: sha256:42c510eff188592c3b76825feff8cb5d13fe00fec0d72d98685ba5e3b26ab8
Status: Downloaded newer image for nginx:alpine
80430b45aaba7260d0ebd27246d1ccb1dec442585685a24ee8486442a130eff
PS C:\Users\Alina>
```

Проверьте, что контейнер работает: Откройте в браузере <http://localhost:8080>.
Вы должны увидеть стартовую страницу

Выполнение



- Остановите и удалите контейнер:

```
docker ps
```

```
docker stop my_webtestservak
```

docker ps -a

docker rm my_webtestservak

Выполнение

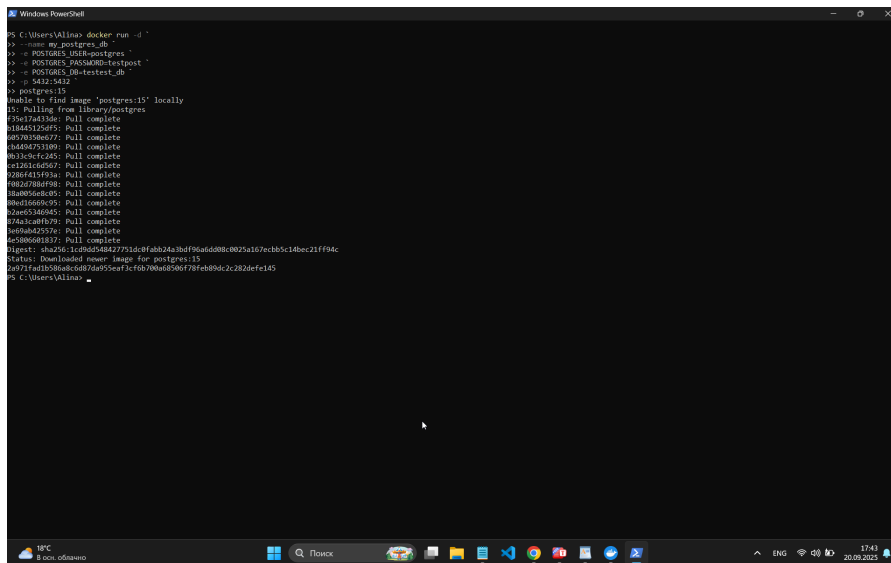
```
Windows PowerShell
403e1f251637: Pull complete
7ab4d6741e18: Pull complete
Digest: sha256:42a516af108852e33b7082d5efb6b5d13fe08fecdc7e98605ba5e3b26aab8
Status: Downloaded newer image for nginx:alpine
8043b845aa8a7320b0b8b27246d1ccbdcc442585685a24eeb486442e130eff
PS C:\Users\Alina> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
8043b845aa8a   nginx:alpine   "/docker-entrypoint..."  5 minutes ago    Up 5 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    my_webtestservak
PS C:\Users\Alina> docker stop my_webtestservak
my_webtestservak
PS C:\Users\Alina> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
8043b845aa8a   nginx:alpine   "/docker-entrypoint..."  6 minutes ago    Exited (0) 24 seconds ago                                my_webtestservak
808c9ffc90f    hello-world    "/hello"                  21 minutes ago    Exited (0) 21 minutes ago    focused_e1bakyan
PS C:\Users\Alina> docker rm my_webtestservak
my_webtestservak
PS C:\Users\Alina>
```

Часть 2: Запуск PostgreSQL в контейнере

Запустите контейнер с PostgreSQL:

```
docker run -d\  
-name my_postgres_db\  
-e POSTGRES_USER=postgres\  
-e POSTGRES_PASSWORD=testpost\  
-e POSTGRES_DB=testtest_db\  
-P 5432:5432\  
postgres:15
```

Выполнение



```
Windows PowerShell  
C:\Users\Alina> docker run -d \  
-name my_postgres_db \  
-e POSTGRES_USER=postgres \  
-e POSTGRES_PASSWORD=testpost \  
-e POSTGRES_DB=testtest_db \  
-p 5432:5432 \  
postgres:15  
Unable to find image 'postgres:15' locally  
15: Pulling from library/postgres  
72617a4536e: Pull complete  
51845125df5: Pull complete  
2b87838e877: Pull complete  
3460475389: Pull complete  
0b33c3cf245: Pull complete  
c2251c5c507: Pull complete  
9286415f93a: Pull complete  
0a2278bf09: Pull complete  
3aa056e8c05: Pull complete  
0a0d0668c05: Pull complete  
2ae0536045: Pull complete  
074a1a0b079: Pull complete  
1d0ba425376: Pull complete  
6e5806081837: Pull complete  
Digest: sha256:1c0b054d27758d8fda3a2a0aff9a6d08b0025a167cbb5c14bec21ff94c  
Status: Downloaded newer image for postgres:15  
d0971ad1d586ac6d87da955eaf3cf02700a68506f78feb9dc2c202defe145  
C:\Users\Alina>
```

Проверьте, что контейнер запущен и слушает порт:

```
docker ps
```

Выполнение

```
Windows PowerShell
PS C:\Users\Allina> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                    NAMES
2a971fd1d158   postgres:15   "docker-entrypoint.sh"   About a minute ago   Up about a minute   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   my_postgres_db
```

Подключитесь к БД прямо из контейнера (через psql):

```
docker exec -it my_postgres_db psql -U postgres -d test_db
```

`docker exec -it my_postgres_db psql -U postgres -d testtest_db`

```
\l          -- Список всех баз данных
\d          -- Список таблиц в текущей БД (пока пусто)
CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(50)); -- Создать таблицу
INSERT INTO users (name) VALUES ('Alice'), ('Bob'); -- Вставить данные
SELECT * FROM users; -- Выбрать данные
\q         -- Выйти из psql
```


Выполнение

```
Windows PowerShell
testest_db=# CREATE TABLE users (id SERIAL PRIMARY KEY, NAME VARCHAR (50));
INSERT INTO users (name) VALUES ('Alica'), ('Bob');
SELECT * FROM users;
ERROR: relation "users" already exists
INSERT 0 2
 id | name
-----+-----
  1 | Alica
  2 | Bob
(2 rows)

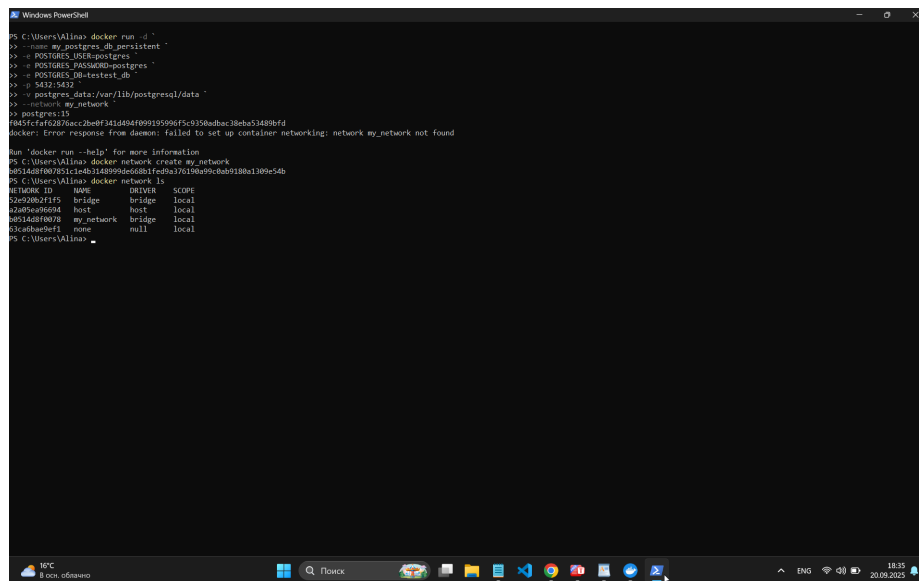
testest_db=# \q
PS C:\Users\Valina>
```

Часть 3: Подключение к БД через pgAdmin из второго контейнера

Создайте сеть Docker: Контейнеры по умолчанию изолированы. Чтобы они "увидели" друг друга по именам, нужна общая сеть.

```
docker network create my_network
docker network ls # Убедитесь, что сеть создана
```

Выполнение:



```
PS C:\Users\Valina> docker run -d `
> --name my_postgres_db persistent `
> -e POSTGRES_USER=postgres `
> -e POSTGRES_PASSWORD=postgres `
> -e POSTGRES_DB=testdb `
> -p 5432:5432 `
> --postgres_data=/var/lib/postgresql/data `
> --network my_network `
> postgres:13
b8514d8f007851c14b314d899d66081f2ba2176190a99c0ab9180a1309e54b
docker: Error response from daemon: failed to set up container networking: network my_network not found

Run 'docker run --help' for more information
PS C:\Users\Valina> docker network create my_network
b8514d8f007851c14b314d899d66081f2ba2176190a99c0ab9180a1309e54b
PS C:\Users\Valina> docker network ls
NETWORK ID          NAME                DRIVER          SCOPE
52e920b2f1f5        bridge              bridge          local
12a0f5a0e604        host                host            local
b8514d8f0078        my_network          bridge          local
93c40ba0ef1         none               null            local
PS C:\Users\Valina>
```

Подключите контейнер с PostgreSQL к сети:

```
docker network connect my_network my_postgres_db
```

Запустите pgAdmin в той же сети:

```
docker run -d \
--name my_pgadmin \
-e PGADMIN_DEFAULT_EMAIL=admin@example.com \
-e PGADMIN_DEFAULT_PASSWORD=admin \
-p 8080:80 \
--network my_network \
dpage/pgadmin4
```

Выполнение

```
Windows PowerShell
PS C:\Users\Allina> docker run -d ^
> --name my_pgadmin ^
> -P PGADMIN_DEFAULT_EMAIL=admin@example.com ^
> -P PGADMIN_DEFAULT_PASSWORD=admin ^
> -p 8080:80 ^
> --network my_network ^
> dpage/pgadmin
Unable to find image 'dpage/pgadmin:latest' locally
latest: Pulling from dpage/pgadmin
8f9d27108efb: Pull complete
8fca3142a7: Pull complete
20f5c90344: Pull complete
2f693c4d3d1: Pull complete
9f2fe11f43b: Pull complete
4aa70aa07d1: Pull complete
7dc5f1c318ba: Pull complete
11a0b7d6eb3: Pull complete
464d366a312b: Pull complete
1310b707921d: Pull complete
f460ff1c5d4b: Pull complete
8eeca1d570f: Pull complete
2d5d02c2ed1: Pull complete
2ba80dc2ab02: Pull complete
Digest: sha256:d12b0ed7f7940a6f6b1a54429d5b0eb850e782e236310219fa761f4022f49
Status: Downloaded newer image for dpage/pgadmin:latest
ac614a2ba7029beac6d97c959705091a2c9edfcaa98d3c1d1cc6aaf712
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint my_pgadmin (0596079b43a17d17dc6c34c4fc58e51beb949306a0f595ef147e688d538b7e
). Bind for 0.0.0.0:8080 failed: port is already allocated

Run 'docker run --help' for more information
PS C:\Users\Allina>
```

Настройте подключение в pgAdmin:

Откройте <http://localhost:8080> в браузере.

Войдите под **admin@example.com / admin**.

Добавьте новый сервер:

General -> Name: Docker PostgreSQL (любое имя)

Connection -> Host name/address: my_postgres_db (имя контейнера с БД!)

Connection -> Username: postgres

Connection -> Password: postgres

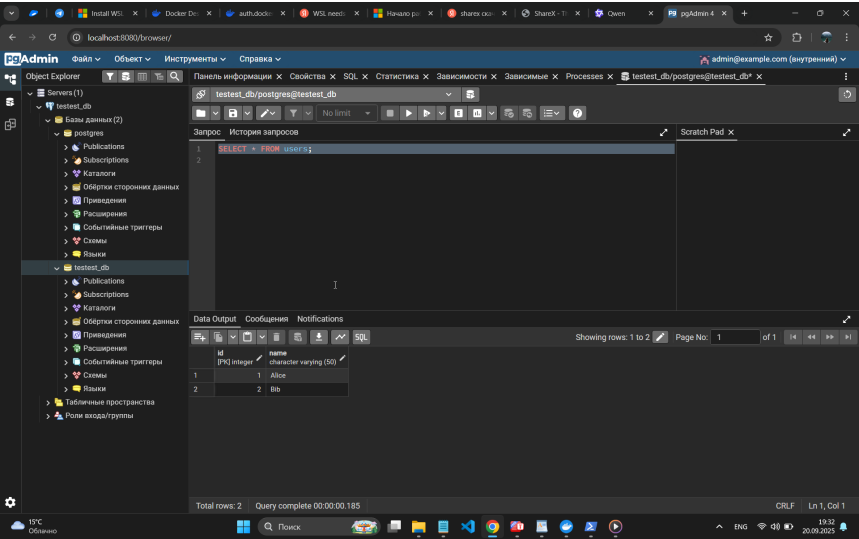
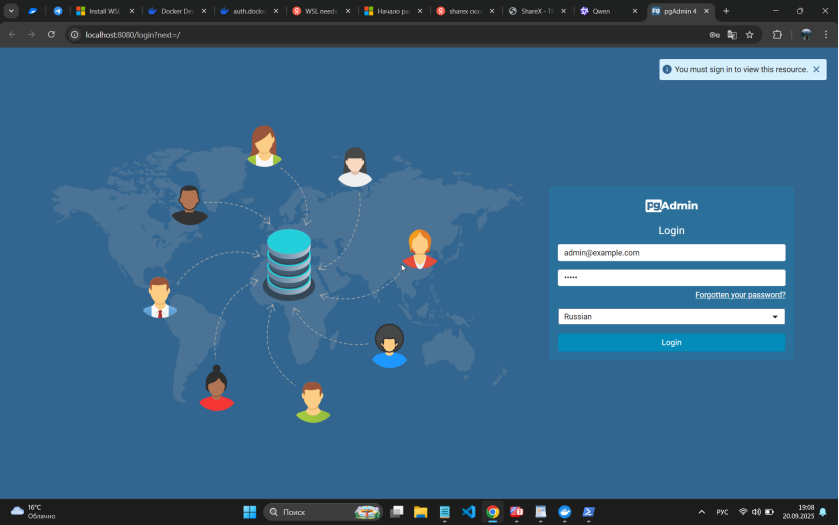
Сохраните. Подключение должно пройти успешно.

Через Query Tool в pgAdmin выполните запрос:

```
SELECT * FROM users;
```

*Вы должны увидеть таблицу **users**, созданную ранее через консоль, и данные в ней.*

Выполнение



Часть 4: Сохранение данных с помощью Томов (Volumes)

Остановите и удалите текущий контейнер с БД:

Выполнение

```
Windows PowerShell
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
2a971fad1b58   postgres:15 "docker-entrypoint.s..." 34 minutes ago Up 34 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp my_postgres_db

PS C:\Users\Valina> docker stop my_postgres_db
docker: unknown command: docker stop

Run 'docker --help' for more information
PS C:\Users\Valina> docker stop my_postgres_db
my_postgres_db

PS C:\Users\Valina> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
2a971fad1b58   postgres:15 "docker-entrypoint.s..." 34 minutes ago Exited (0) 30 seconds ago          my_postgres_db
bdc5ffc280f    hello-world "hello"              About an hour ago Exited (0) About an hour ago          focused_elbakyan

PS C:\Users\Valina> docker rm my_postgres_db
my_postgres_db

PS C:\Users\Valina> docker rm focused_elbakyan
focused_elbakyan

PS C:\Users\Valina> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
PS C:\Users\Valina>
```

- Создайте том для хранения данных БД:

```
docker volume create postgres_data
docker volume ls
```

```
docker volume create postgres_data
```

```
docker volume ls
```

Выполнение

```
Windows PowerShell
focused@hakeyan
PS C:\Users\Valina> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
PS C:\Users\Valina> docker volume create postgres_data
postgres_data
PS C:\Users\Valina> docker volume ls
DRIVER    VOLUME NAME
local    673b97e6d2b2ff480b43cad707a357a1891568a07df9fdeebd0213aca811bf9
local    postgres_data
PS C:\Users\Valina>
```

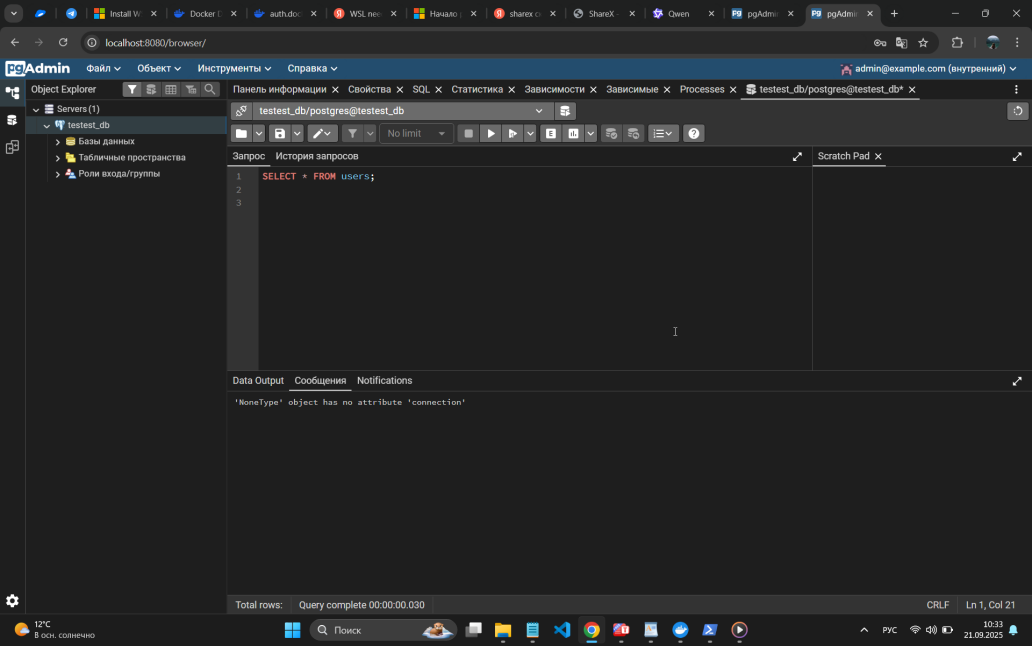
Запустите новый контейнер с PostgreSQL, подключив том:

```
docker run -d \
  --name my_postgres_db_persistent \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=test_db \
  -p 5432:5432 \
  -v postgres_data:/var/lib/postgresql/data \ # Ключевая строка!
  --network my_network \ # Подключаем его в ту же сеть для pgAdmin
  postgres:15
```

Проверьте сохранность данных:

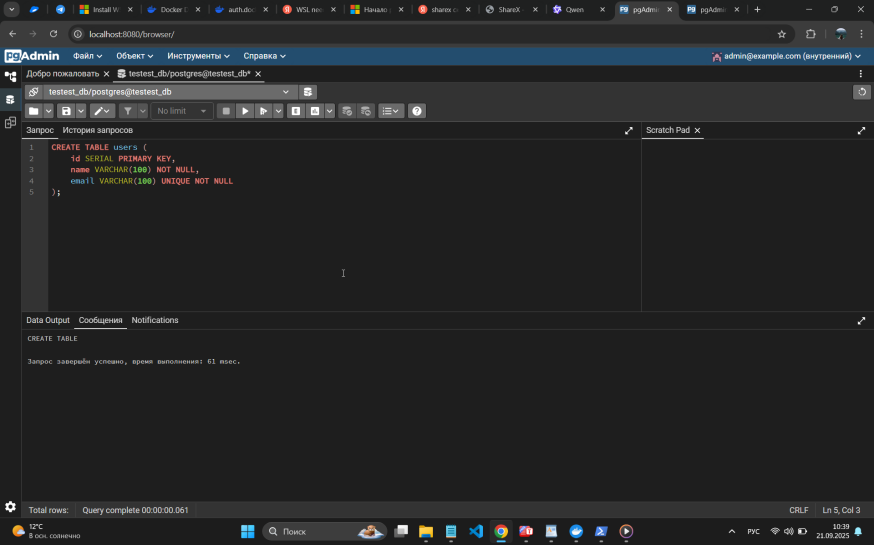
Через pgAdmin (<http://localhost:8080>) снова выполните запрос **SELECT * FROM users;**

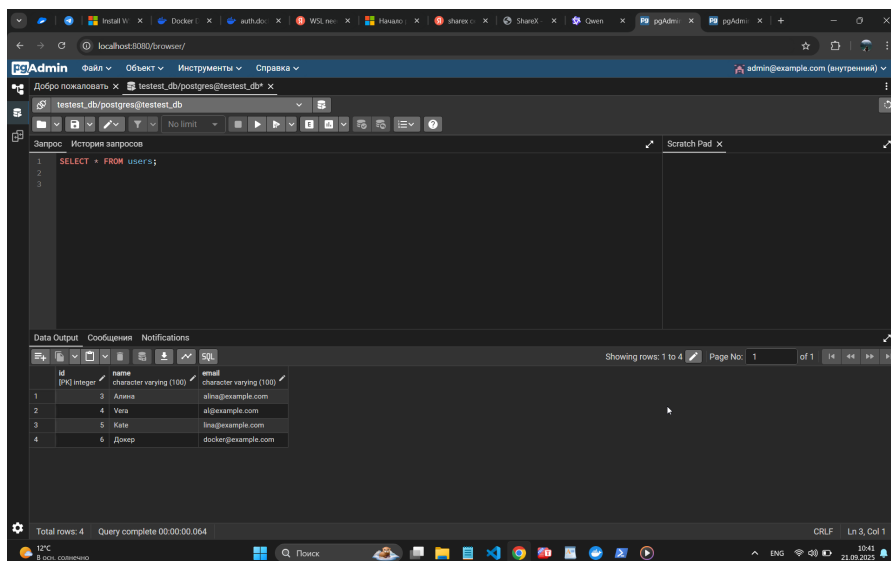
Выполнение



Добавьте таблицу и данные

Выполнение





Остановите контейнеры с БД и pgAdmin

Запустите контейнеры

```
docker start my_postgres_db my_pgadmin
```

Проверьте таблицу users

Выполнение

```
Windows PowerShell
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
3d18a2f9ef1   dpag/pgadmin4  "/entrypoint.sh"        16 hours ago  Up 16 hours  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  my_pgadmin
f045fcfa628   postgres:15   "docker-entrypoint.s..." 16 hours ago  Up 12 minutes  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  my_postgres_db_persistent

PS C:\Users\Alina> docker stop my_pgadmin
my_pgadmin
PS C:\Users\Alina> docker stop my_pgadmin my_postgres_db_persistent
my_pgadmin
my_postgres_db_persistent
PS C:\Users\Alina>
```

12°C
В осн. солнечно

10:44
21.09.2025

```
Windows PowerShell
my_postgres_db_persistent
Error: failed to start containers: my_postgres_db
PS C:\Users\Alina> docker start my_pgadmin my_postgres_db_persistent
my_pgadmin
my_postgres_db_persistent
PS C:\Users\Alina> docker exec -it my_postgres_db_persistent psql -U postgres -d testtest_db
psql (15.14 (Debian 15.14-1.pgdg13+1))
Type "help" for help.

testtest_db=# SELECT * users;
ERROR: syntax error at or near "users"
LINE 1: SELECT * users;
              ^
testtest_db=# SELECT *from users;
 id | name | email
-----+-----+-----
  3 | Ална | alina@example.com
  4 | Vera | al@example.com
  5 | Kate | lina@example.com
  6 | Докеп | docker@example.com
(4 rows)

testtest_db=#
```

12°C
В осн. солнечно

10:50
21.09.2025

Часть 5: Перенос конфигурации контейнеров в docker-compose.yaml

После настройки pgAdmin и PostgreSQL настраиваем docker-compose.yaml

Дополнительная информация: <https://docs.docker.com/compose/>

Выполнение

services:

my_postgres_db_persistent:

image: postgres:15

container_name: my_postgres_db_persistent

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: testest_db

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

networks:

- my_network

Сервис pgAdmin

my_pgadmin:

image: dpage/pgadmin4

container_name: my_pgadmin

environment:

PGADMIN_DEFAULT_EMAIL: admin@example.com

PGADMIN_DEFAULT_PASSWORD: admin

ports:

- "8080:80"

depends_on:

- my_postgres_db_persistent

networks:

- my_network

volumes:

postgres_data:

networks:

my_network:

Ответы на вопросы

Что такое docker?

Docker — это платформа для разработки, доставки и запуска приложений в изолированных средах, называемых *контейнерами*. Позволяет легко масштабировать и комбинировать разные сервисы.

Docker позволяет упаковать приложение один раз и запускать его где угодно.

Примеры:

- `hello-world` — простой контейнер для проверки установки.
- `nginx:alpine` — веб-сервер в контейнере.
- `postgres:15` — полноценная база данных PostgreSQL, запущенная одной командой.
- `dpage/pgadmin4` — графический интерфейс для управления PostgreSQL.

Для чего нужны тома и сети docker?

Тома и сети нужны для **сохранения данных и взаимодействия между сервисами**. Тома (**Volumes**) нужны для **постоянного хранения данных**. Они позволяют сохранить данные независимо от жизненного цикла контейнера.

Почему это важно: По умолчанию все данные внутри контейнера *эфемерны* — они исчезают, когда контейнер удаляется. Тома позволяют сохранять данные (например, таблицы и записи вашей базы данных PostgreSQL) **независимо от жизненного цикла контейнера**.

Например, команда `volumes: - postgres_/var/lib/postgresql/data`

Данная строка позволит сохранить данные из контейнера `my_postgres_db_persistent` в томе `postgres_data`, если придется удалить `my_postgres_db_persistent`. При запуске нового контейнера можно будет подключить к нему этот же том.

Сети (Networks) нужны для того, чтобы контейнеры могли безопасно и предсказуемо общаться друг с другом.. Сеть позволяет им находить друг друга по *имени*, а не по IP-адресу.

Например, networks: - my_network, - была создана сеть my_network и подключены контейнеры my_postgres_db_persistent и my_pgadmin. Теперь pgAdmin может подключиться к PostgreSQL, указав в качестве хоста имя контейнера my_postgres_db_persistent.

Как подключиться к контейнеру и выполнить в нём команды?

Для этого используется команда docker exec.

```
docker exec -it <имя_или_ID_контейнера> <команда>
```

Подключение к PostgreSQL для выполнения SQL-запросов: docker exec -it my_postgres_db_persistent psql -U postgres -d testtest_db - Эта команда запускает внутри контейнера psql (клиент PostgreSQL) и подключается к вашей базе данных.

Запуск интерактивной оболочки (например, bash) внутри контейнера: docker exec -it my_postgres_db_persistent bash - Это "открывает терминал" прямо внутри контейнера. Оттуда вы можете исследовать файловую систему, запускать любые команды Linux и т.д.

Для чего нужен pgAdmin?

pgAdmin — это *графический веб-интерфейс* для администрирования и разработки с PostgreSQL.

- **Удобен и понятен в использовании:** Не нужно помнить SQL-команды для создания таблиц или просмотра структуры. Всё можно сделать через интуитивно понятные меню и формы.
- **Редактор запросов:** Мощный редактор с подсветкой синтаксиса, автодополнением и историей запросов.
- **Импорт/экспорт:** Удобные инструменты для загрузки и выгрузки данных.
- **Мониторинг:** Просмотр активных сессий, производительности и журналов.

В своей работе поризвели запуск pgAdmin в отдельном контейнере и подключили его к контейнеру с PostgreSQL через сеть Docker. В результате получили возможность управлять базой данных через браузер по адресу <http://localhost:8080>.

Вывод

В ходе выполнения лабораторной работы были освоены ключевые концепции и инструменты платформы Docker. Была создана и протестирована полностью изолированная и воспроизводимая среда, состоящая из веб-сервера, базы данных и графического интерфейса для её администрирования. Использование томов гарантирует сохранность данных при перезапуске или удалении контейнеров, а применение пользовательских сетей обеспечивает надёжное и удобное взаимодействие сервисов по их именам. Файл `docker-compose.yaml` позволяет развернуть всю инфраструктуру одной командой, что является стандартом современной разработки и DevOps-практик. Полученные навыки являются фундаментальными для дальнейшей работы с микросервисной архитектурой и облачными технологиями.